Practical No.	Date	Practical Aim	Signature
1		**Image Enhancement in Spatial Domain** a) Log Transformation b) Power Law Transformation c) Negation d) Contrast Stretching (Dark, Bright, Low Contrast and High Contrast) e) Gray-Level Slicing with and without Background f) Bit-Plane Slicing	
2		**Image Enhancement in Frequency Domain** a) Discrete Fourier Transform (DFT) b) Gaussian Low Pass Filter c) Butterworth Low Pass Filter d) Ideal High Pass Filter e) Ideal Low Pass Filter	
3		**Image Enhancement in Spatial Domain** a) Image Averaging for Noise Removal Using Averaging Filter b) Smoothing c) Varying the Number of Intensity Levels in a Digital Image d) Image Averaging (Image Addition) for Noise Reduction e) Image Subtraction for Enhancing the Difference Between Images	
4		**Image Enhancement Statistical Filters** a) Mean Filter and Median Filter b) Max-Min and Min-Max Filters	
5		**Morphological Operations** a) Erosion and Dilation b) Opening and Closing c) Boundary Extraction d) Morphological Gradient e) Top-Hat Transformation	
6		**Region Filling** Fill the Empty Region Inside an Image Using Region Filling Procedure	
7		**OpenCV** a) Install and Import OpenCV ii) b) Read, Write and Display an Image c) Grayscale Morphological Operations d) Image Blurring	
8		**Video Processing**	
9		**Color Image Processing** a) Read a Color Image and Segment It into RGB Planes, Display Color Image Histogram b) Convert an Image from One Color Model to Another Color Model c) Apply False Colouring (Pseudo Colouring) on a Grayscale Image	

PRACTICAL NO 1: Image Enhancement in Spatial Domain

A. Log Transformation

Aim: To enhance an image using log transformation.

Code:

```
clc;
clear;
close;
a = imread('C:\ip\img.png');
a = double(rgb2gray(a));
c = 255 / log(256);
b = c * log(1 + a);
b = uint8(b);
imshow(b);
title("Log Transformation");
```

B. Power Law Transformation

Aim: To enhance an image using power law transformation.

Code:

```
clc;
clear all;
a = imread('C:\ip\images.jpg');
b = double(a);
subplot(1,2,1);
imshow(a);
title('Original img');
k=1
gamma=1;
[m,n] = size(b);
for i = 1:m
   for j = 1:n
      c(i,j)=k*(b(i,j)^gamma);
   end
end
subplot(1,2,2);
imshow(uint8(c));
title('Power Law img');
```

C. Negation
Aim: To obtain the negative of an input image.

Code:

```
clc;
clear all;
a = imread('C:\ip\images.jpg');
subplot(1,2,1);
imshow(a);
title('Original img');
[m,n] = size(a);
for i = 1:m
   for j = 1:n
      c(i,j) = 255 - a(i,j);
   end
end
subplot(1,2,2);
imshow(c);
title('Negation img');
```

D. Contrast Stretching
Aim: To generate dark, bright, low-contrast, and high-contrast images.

Code:

```
clc;
clear all;

a = imread('C:\ip\images.jpg');
subplot(2,3,1);
imshow(a);
title('Original Image');

dark = a * 0.4;
subplot(2,3,2);
imshow(dark);
title('Dark Image');

bright = a * 1.5;
bright(bright > 1) = 1;
subplot(2,3,3);
imshow(bright);
title('Bright Image');

low = 0.5 * a + 0.25;
subplot(2,3,4);
imshow(low);
title('Low Contrast Image');
```

```
high = imadjust(a, [0.3 0.7], [0 1]);
subplot(2,3,5);
imshow(high);
title('High Contrast Image');
```

E. Gray-Level Slicing With and Without Background
Aim: To highlight a selected range of gray levels in an image.

Code:

```
clc;
clear all;

a = imread('C:\ip\image.jpg');
a = rgb2gray(a);

[m,n] = size(a);

b = zeros(m,n);
c = a;

for i = 1:m
   for j = 1:n
      if a(i,j) >= 100 & a(i,j) <= 200 then
         b(i,j) = 255;
         c(i,j) = 255;
      end
   end
end

subplot(1,3,1);
imshow(a);
xtitle("Original Image");

subplot(1,3,2);
imshow(uint8(b));
xtitle("Without Background");

subplot(1,3,3);
imshow(c);
xtitle("With Background");
```

F. Bit-Plane Slicing
Aim: To separate an image into its individual bit planes.

.

Code:

```
clc;
clear all;
close all;

a = imread('C:\ip\images.jpg');
a = rgb2gray(a);

subplot(3,3,1);
imshow(a);
title('Original Image');

for k = 0:7
    b = bitget(a,k+1) * 255;
    subplot(3,3,k+2);
    imshow(b);
    title(['Bit Plane ',num2str(k)]);
end
```

Practical 2: Image Enhancement in Frequency Domain

A. Discrete Fourier Transform (DFT)
Aim: To apply the Discrete Fourier Transform on an image.

Code:

```
clc;
clear all;

I = imread("C:\ip\images.jpg");
I1 = double(I);
J = fft2(I1);
L = fftshift(real(J));

subplot(1,3,1);
imshow(I);
title('Original Image');

subplot(1,3,2);
imshow(J);
title('Fourier transform');

subplot(1,3,3);
imshow(L);
title('shifted Fourier transform');
```

B. Gaussian Low Pass Filter
Aim: To smooth an image using a Gaussian low-pass filter.
Code:

```
clc;
clear all;

S = imread("C:\ip\images.jpg");
S = rgb2gray(S);
S2 = fft2(im2double(S));
S3 = fftshift(S2);

[M,N] = size(S);
[u,v] = meshgrid(1:N,1:M);
D = sqrt((u-N/2).^2 + (v-M/2).^2);
D0 = 30;

H = exp(-(D.^2)/(2*D0^2));
S4 = real(ifft2(ifftshift(S3.*H)));

subplot(1,3,1);
```

```
imshow(S);
title('Original Image');

subplot(1,3,2);
imshow(log(1+abs(S3)),[]);
title('Fourier Transform');

subplot(1,3,3);
imshow(S4,[]);
title('Gaussian Low Pass Filter');
```

C. Butterworth Low Pass Filter
Aim: To apply a Butterworth low-pass filter to an image.
Code:
clc;

```
clear all;

S = imread("C:\ip\images.jpg");
h = mkfftfilter(S,'gauss',0.4);
S2 = fft2(im2double(S));
S3 = S2.*fftshift(h);
```

```
S4 = real(ifft(S3));

subplot(2,2,1);
imshow(S);
title('Original Image');

subplot(2,2,2);
imshow(abs(S2));
title('Fourier Transform');

subplot(2,2,3);
imshow(abs(S3));
title('Shifted Fourier Transform');

subplot(2,2,4);
imshow(S4);
title('Filtered Image');
```

D. Ideal High Pass Filter
Aim: To apply an ideal high-pass filter to an image.

Code:

```
clc;
clear all;

S = imread("C:\ip\images.jpg");
S2 = fft2(im2double(S));
h = ffilt('hp',3,0.2,0.3);
img = imfilter(S2,h);
S4 = real(ifft(img));

subplot(1,3,1);
imshow(S);
title('Original Image');

subplot(1,3,2);
```

```
imshow(abs(S2));
title('Fourier Image');

subplot(1,3,3);
imshow(S4);
title('Filtered Image');
```

E. Ideal Low Pass Filter
Aim: To apply an ideal low-pass filter to an image.

Code:

```
clc;
clear all;

S = imread("C:\ip\images.jpg");
S2 = fft2(im2double(S));
h = ffilt('lp',3,0.2,0.3);
img = imfilter(S2,h);
S4 = real(ifft(img));

subplot(1,3,1);
imshow(S);
title('Original Image');

subplot(1,3,2);
imshow(abs(S2));
title('Fourier Image');

subplot(1,3,3);
imshow(S4,[]);
title('Filtered Image');
```

Practical 3: Image Enhancement in Spatial Domain

A. Image Averaging Using Averaging Filter

Aim: To remove noise from an image using an averaging filter.

Code:

```
clc;
clear all;

image = imread("C:\ip\image.jpg");
NoisyImage = imnoise(image,'salt & pepper',0.02);

F1 = fspecial('average',3);
FilterImage = imfilter(NoisyImage,F1);

subplot(1,3,1);
imshow(image);
title('Original Image');

subplot(1,3,2);
imshow(NoisyImage);
title('Noisy Image');

subplot(1,3,3);
imshow(FilterImage);
title('Filtered Image');
```

B. Smoothing

Aim: To apply a smoothing effect to an input image using an appropriate filter.

Code:

```
clc;
clear all;

a = imread("C:\ip\image.jpg");
a = rgb2gray(a);

NoisyImage = imnoise(a,'gaussian',0.02);
wienerfilter = imwiener2(NoisyImage,[5,5],0.02);

subplot(1,3,1);
imshow(a);
title('Original Image');

subplot(1,3,2);
imshow(NoisyImage);
title('Noisy Image');

subplot(1,3,3);
```

```
imshow(wienerfilter);
title('Wiener Filter');
```

C. Varying the Number of Intensity Levels
Aim: To study the effects of varying the number of intensity levels in a digital image.
Code:

```
clc;
clear;

a = imread("C:\ip\image.jpg");
a = double(rgb2gray(a));

subplot(2,2,1);
imshow(uint8(a));
xtitle("Original Image");

subplot(2,2,2);
imshow(uint8(floor(a/128)*128));
xtitle("2 Intensity Levels");

subplot(2,2,3);
imshow(uint8(floor(a/64)*64));
xtitle("4 Intensity Levels");

subplot(2,2,4);
imshow(uint8(floor(a/32)*32));
xtitle("8 Intensity Levels");
```

D. Image Averaging (Image Addition) for Noise Reduction
Aim: To perform image averaging using image addition for noise reduction.

Code:

```
clc;
clear;

a = double(imread("C:\ip\image.jpg"));
b = double(imread("C:\ip\image.jpg"));

avg = uint8((a+b)/2);

subplot(1,3,1);
imshow(uint8(a));
xtitle("Image 1");

subplot(1,3,2);
imshow(uint8(b));
xtitle("Image 2");

subplot(1,3,3);
imshow(avg);
xtitle("Averaged Image");
```

E. Image Subtraction
Aim: To compare images using subtraction and enhance the difference between them.

Code:

```
clc;
clear;

a = double(imread("C:\ip\image.jpg"));
b = double(imread("C:\ip\image.jpg"));

c = uint8(abs(a-b));

subplot(1,3,1);
imshow(uint8(a));
xtitle("Image 1");

subplot(1,3,2);
imshow(uint8(b));
xtitle("Image 2");

subplot(1,3,3);
imshow(c);
xtitle("Subtracted Image");
```

Practical 4: Image Enhancement Statistical Filters

A. Mean Filter

Aim: To apply a mean filter to an input image.

Code:

```
clc;
clear;
close;

img = imread("D:\gray.jpeg");
img = double(img);
[m, n] = size(img);
filtered = zeros(m, n);

for i = 2:m-1
  for j = 2:n-1
     window = img(i-1:i+1, j-1:j+1);
     filtered(i,j) = sum(window) / 9;
  end
end

filtered = uint8(filtered);

figure();
subplot(1,2,1);
imshow(uint8(img));
title("Original Image");

subplot(1,2,2);
imshow(filtered);
title("Mean Filtered Image");
```

B. Median Filter
Aim: To apply a median filter to an input image.

Code:

```
clc;
clear;
close;

img = imread("D:\gray.jpeg");
img = double(img);

[m, n] = size(img);
filtered = img;

for i = 2:m-1
  for j = 2:n-1
     window = img(i-1:i+1, j-1:j+1);
     pixels = matrix(window, 1, 9);
     filtered(i,j) = median(pixels);
  end
end

filtered = uint8(filtered);

figure();
subplot(1,2,1);
imshow(uint8(img));
title("Original Image");

subplot(1,2,2);
imshow(filtered);
title("Median Filtered Image");
```

C. Max-Min and Min-Max Filters

Aim: To apply max-min and min-max filters to an input image.

Code:

```
clc;
clear;
close;

img = imread("D:\gray.jpeg");
img = double(img);

[m, n] = size(img);
max_filtered = img;
min_filtered = img;
maxmin_filtered = img;

for i = 2:m-1
  for j = 2:n-1
     window = img(i-1:i+1, j-1:j+1);

     max_filtered(i,j) = max(window);
     min_filtered(i,j) = min(window);

     maxmin_filtered(i,j) = (max(window) + min(window)) / 2;
  end
end

max_filtered = uint8(max_filtered);
min_filtered = uint8(min_filtered);
maxmin_filtered = uint8(maxmin_filtered);

figure();
subplot(2,2,1);
imshow(uint8(img));
title("Original Image");

subplot(2,2,2);
imshow(max_filtered);
title("Max Filter");
```

```
subplot(2,2,3);
imshow(min_filtered);
title("Min Filter");

subplot(2,2,4);
imshow(maxmin_filtered);
title("Max-Min Filter");
```

Practical 5: Morphological Operations

A. Erosion

Aim: To perform erosion on an input image..

Code:

```
clc;
clear all;

a = imread('E:\IMGPROCESSING\erosion.png');

structEle = imcreatese('ellipse',5,5);

erosion = imerode(a,structEle);

subplot(1,2,1);
imshow(a);
title('Original Image');

subplot(1,2,2);
imshow(erosion);
title('Eroded Image 1');
```

B. Dilation

Aim: To perform dilation on an input image.

Code:

```
clc;
clear all;

a = imread('E:\IMGPROCESSING\erosion.png');

structEle = imcreatese('ellipse',15,15);

dilation = imdilate(a,structEle);

subplot(1,2,1);
imshow(a);
title('Original Image');

subplot(1,2,2);
imshow(dilation);
title('Dilated Image 1');
```

C. Opening
Aim: To perform opening on an input image.
Code:

```
clc;
clear all;

I = imread("C:\ip\images.png");

sturctEle = imcreatese('ellipse',20,20);
erosion = imerode(I,sturctEle);
opening = imdilate(erosion,sturctEle);

subplot(1,2,1);
imshow(I);
title('Original Image');

subplot(1,2,2);
imshow(opening);
title('Opening Image');
```

D. Closing
Aim: To perform closing on an input image.
Code:

```
clc;
clear all;

I = imread("C:\ip\image.JPG");

sturctEle = imcreatese('ellipse',20,20);
dilation = imdilate(I,sturctEle);
closing = imerode(dilation,sturctEle);

subplot(1,2,1);
imshow(I);
title('Original Image');

subplot(1,2,2);
imshow(closing);
title('Closing Image');
```

E. Boundary Extraction
Aim: To extract the boundary of an object in an image.

Code:

```
clc;
clear all;

I = imread("C:\ip\images.png");

sturctEle = imcreatese('ellipse',30,30);
erosion = imerode(I,sturctEle);

boundry = I - erosion;

subplot(2,2,1);
imshow(I);
title('Original Image');

subplot(2,2,2);
imshow(erosion);
title('Eroded Image');

subplot(2,2,3);
imshow(boundry);
title('Boundary Image');
```

F. Morphological Gradient
Aim: To perform the morphological gradient operation on an image.
Code:

```
clc;
clear all;

I = imread("C:\ip\images.png");

sturctEle = imcreatese('ellipse',30,30);
dilation = imdilate(I,sturctEle);
erosion = imerode(I,sturctEle);

gradient = dilation - erosion;

subplot(2,2,1);
imshow(I);
title('Original Image');

subplot(2,2,2);
imshow(dilation);
title('Dilated Image');

subplot(2,2,3);
```

```
imshow(erosion);
title('Eroded Image');

subplot(2,2,4);
imshow(gradient);
title('Morphological Gradient');
```

G. Top-Hat Transformation
Aim: To perform top-hat transformation on an input image.
Code:

```
clc;
clear all;

I = imread("C:\ip\images.png");

sturctEle = imcreatese('ellipse',30,30);
erosion = imerode(I,sturctEle);
opening = imdilate(erosion,sturctEle);

tophat = I - opening;

subplot(2,2,1);
imshow(I);
title('Original Image');

subplot(2,2,2);
imshow(opening);
title('Opening Image');

subplot(2,2,3);
imshow(tophat);
title('Top-hat Image');
```

Practical 6: Region Filling

Aim: To fill the empty region inside an image using the region-filling procedure.

Code:

```
clc;
clear;

S = imread("C:\ip\images.jpg");
S = rgb2gray(S);
BW = im2bw(S);

subplot(1,3,1);
imshow(S);
xtitle("Original Image");

subplot(1,3,2);
imshow(BW);
xtitle("Binary Image");

Filled = imfill(BW, "holes");

subplot(1,3,3);
imshow(Filled);
xtitle("Region Filled Image");
```

Practical 7: OpenCV Python Practicals

A. Read and Display Image
Aim: To read and display an image using OpenCV in Python..

Code:

```
import cv2
import numpy as np

img = cv2.imread("C:\\IPMSIT\\img1.jpg", 1)
cv2.imshow("image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

B. Accessing and Modifying Pixel
Aim: To access and display a pixel value from an image.

Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg', 1)
cv2.imshow('image:', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
pixel = img[100,100]
print(pixel)
```

C. Image Properties
Aim: To display the properties of an image using OpenCV.

Code:

```
import cv2

img = cv2.imread('C:\\IPMSIT\\img1.jpg', 1)
dimensions = img.shape
height = img.shape[0]
width = img.shape[1]
channels = img.shape[2]
size1 = img.size

print('Image Dimension:', dimensions)
print('Image Height:', height)
print('Image Width:', width)
print('Number of Channels:', channels)
print('Image Size:', size1)
```

D. Change Image Color to Grayscale
Aim: To convert a color image into a grayscale image using OpenCV.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\i.jpg', 1)
cv2.imshow('Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

window_name = 'Image'
image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow(window_name, image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

E. Rotation by 90 Degrees
Aim: To rotate an image by 90 degrees using OpenCV.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img1.jpg', 1)
cv2.imshow('image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

(h,w) = img.shape[:2]
center = (w/2,h/2)
angle90 = 90
scale = 1.0
M = cv2.getRotationMatrix2D(center,angle90,scale)
rotated90 = cv2.warpAffine(img,M,(h,w))

cv2.imshow('Image rotated by 90 degrees',rotated90)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

F. Draw a Circle

Aim: To draw a circle on an image using OpenCV.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
cv2.circle(img,(80,80),55,(0,255,0),-1)
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

G. Draw a Square / Rectangle
Aim: To draw a rectangle on an image using OpenCV.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
cv2.rectangle(img,(15,25),(200,150),(0,255,255),15)
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

H. Draw an Ellipse
Aim: To draw an ellipse on an image using OpenCV.

Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
cv2.ellipse(img,(250,150),(80,20),5,0,360,(0,0,255),15)
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

I. Draw a Line
Aim: To draw a line on an image using OpenCV.

Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
cv2.line(img,(10,0),(150,150),(0,0,255),15)
cv2.imshow('image',img)
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

J. Add Text
Aim: To add text to an image using OpenCV.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img,'HELLO',(150,150),font,1,(0,0,255))
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

K. Edge Detection
Aim: To detect edges in an image using the Canny edge detector.

Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img2.jpg',1)
cv2.imshow('image',img)
edges = cv2.Canny(img,100,200)
cv2.imshow("Edge Detected Image",edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

L. OpenCV Blur (Image Smoothing)
Aim: To smooth an image using the OpenCV blur function.
Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img5.jpg',1)
cv2.imshow('image',cv2.blur(img,(5,5)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

M. Image Thresholding
Aim: To convert an image into a binary image using thresholding.

Code:

```
import cv2
import numpy as np

img = cv2.imread('C:\\IPMSIT\\img1.jpg',1)
retval,threshold = cv2.threshold(img,62,255,cv2.THRESH_BINARY)
cv2.imshow('Image',threshold)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Practical 8: Video Processing

A. Gaussian Blur

Aim: To apply Gaussian blur to video frames using OpenCV.

Code:

```
import cv2
import numpy as np

cap = cv2.VideoCapture('C:\\IPMSIT\\v1.mp4')

while(cap.isOpened()):
    ret, frame = cap.read()
    frame = cv2.resize(frame, (540, 380), fx=0, fy=0,
                       interpolation=cv2.INTER_CUBIC)
    cv2.imshow('Frame', frame)
    gaussianblur = cv2.GaussianBlur(frame, (5, 5), 0)
    cv2.imshow('gblur', gaussianblur)
    if cv2.waitKey(25) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

B. Video Frame Capture

Aim: To capture and display video frames using OpenCV.

Code:

```
import cv2

vid = cv2.VideoCapture(0)

while(True):
    ret, frame = vid.read()
    cv2.imshow('frame', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

vid.release()
cv2.destroyAllWindows()
```

C. Video Thresholding
Aim: To apply adaptive thresholding to video frames.

Code:

```
import cv2
import numpy as np

cap = cv2.VideoCapture('C:\\IPMSIT\\v1.mp4')

while cap.isOpened():
   ret, frame = cap.read()
   if not ret:
      break

   frame = cv2.resize(frame, (540, 380), fx=0, fy=0,
                  interpolation=cv2.INTER_CUBIC)
   cv2.imshow("Frame", frame)
   gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
   Thresh = cv2.adaptiveThreshold(gray, 255,
         cv2.ADAPTIVE_THRESH_MEAN_C,
         cv2.THRESH_BINARY_INV, 11, 2)
   cv2.imshow("Thresh", Thresh)

   if cv2.waitKey(25) & 0xFF == ord('q'):
      break

cap.release()
cv2.destroyAllWindows()
```

D. Image Properties and Image-to-Video Generation
Aim: To resize images to a common size and generate a video from them.
Code:

```
import os
import cv2
from PIL import Image

path = 'C:\\IPMSIT'
os.chdir(path)

mean_height = 0
mean_width = 0
```

```
num_of_images = len(os.listdir('.'))

for file in os.listdir('.'):
    try:
        im = Image.open(os.path.join(path, file))
        width, height = im.size
        mean_width += width
        mean_height += height
    except:
        continue

mean_width = int(mean_width / num_of_images)
mean_height = int(mean_height / num_of_images)

for file in os.listdir('.'):
    if file.endswith(".jpg") or file.endswith(".jpeg") or file.endswith(".png"):
        im = Image.open(os.path.join(path, file))
        imResize = im.resize((mean_width, mean_height))
        imResize.save(file, 'JPEG', quality=95)
        print(file, "is resized")

def generate_video():
    image_folder = path
    video_name = 'mygeneratedvideo.avi'
    os.chdir(image_folder)

    images = [img for img in os.listdir(image_folder)
              if img.endswith(".jpg") or img.endswith(".jpeg") or img.endswith(".png")]

    print("Images found:", images)

    frame = cv2.imread(os.path.join(image_folder, images[0]))
    height, width, layers = frame.shape
    video = cv2.VideoWriter(video_name, 0, 1, (width, height))

    for image in images:
        video.write(cv2.imread(os.path.join(image_folder, image)))

    cv2.destroyAllWindows()
    video.release()
    print("Video generation complete!")

generate_video()
```

Practical 9: Color Image Processing

A. Read a Color Image and Segment It into RGB Planes

Aim: To read a color image and separate it into Red, Green, and Blue (RGB) planes.

Code:

```
clc;
clear all;

I = imread('E:\IP Prac\original.jpg');

r = size(I,1);
c = size(I,2);

R = zeros(r,c,3);
G = zeros(r,c,3);
B = zeros(r,c,3);

R(:,:,1) = I(:,:,1);
G(:,:,2) = I(:,:,2);
B(:,:,3) = I(:,:,3);

subplot(2,2,1);
imshow(I);
title('Original Image');

subplot(2,2,2);
imshow(uint8(R));
title('Red Component');

subplot(2,2,3);
imshow(uint8(G));
title('Green Component');

subplot(2,2,4);
imshow(uint8(B));
title('Blue Component');
```

B. Convert an Image from One Color Model to Another

i. RGB to HSV

Aim: To convert an RGB image into the HSV color model.

Code:

```
clc;
clear all;

rgb = imread('E:\IP Prac\original.jpg');
```

```
hsv = rgb2hsv(rgb);

subplot(2,3,1);
imshow(rgb);
title('Original RGB Image');

subplot(2,3,2);
imshow(hsv);
title('HSV Image');

subplot(2,3,3);
imshow(hsv(:,:,3));
title('Intensity Component');

subplot(2,3,4);
imshow(hsv(:,:,1));
title('Hue Component');

subplot(2,3,5);
imshow(hsv(:,:,2));
title('Saturation Component');
```

ii. RGB to YCbCr
Aim: To convert an RGB image into the YCbCr color model.

Code:

```
clc;
clear all;

rgb = imread('E:\IP Prac\original.jpg');
ycbcr = rgb2ycbcr(rgb);
```

```
subplot(2,3,1);
imshow(rgb);
title('Original RGB Image');

subplot(2,3,2);
imshow(ycbcr);
title('YCBCR Image');

subplot(2,3,3);
imshow(ycbcr(:,:,1));
title('Luminance');

subplot(2,3,4);
imshow(ycbcr(:,:,2));
title('Chrominance Cb');

subplot(2,3,5);
imshow(ycbcr(:,:,3));
title('Chrominance Cr');
```

C. False Colouring (Pseudo Colouring) on a Grayscale Image
Aim: To apply pseudo colouring to a grayscale image.

Code:

```
clc;
clear all;

RGB = imread('E:\IP Prac\Original.jpg');
```

```
Image = rgb2gray(RGB);

subplot(1,3,1);
imshow(RGB);
title('Original RGB Image');

subplot(1,3,2);
imshow(Image);
title('Gray RGB Image');

subplot(1,3,3);
imshow(Image, jetcolormap(256));
title('Pseudo Color Image');

Histogram = imhist(Image);

figure();
plot(0:255, Histogram);
```